

Commande RST de la balle sur poutre

Ce notebook synthétise et simule un correcteur **RST polynomial** pour le modèle de **balle sur poutre**.

Principe du correcteur RST (2-DDL)

Le correcteur RST est un régulateur à **deux degrés de liberté** (2-DDL) dont la loi de commande s'écrit :

$$u[k] = T(z)r[k] - R(z)y[k]$$

- $R(z)$ agit sur la mesure y — fixe les pôles de la boucle fermée.
- $S(z)$ est le dénominateur du correcteur — peut contenir un facteur $(z-1)$ pour l'action intégrale.
- $T(z)$ agit sur la consigne r — ajuste le gain de poursuite.

Les polynômes R , S , T sont calculés par **résolution de l'équation diophantienne** :

$$A(z)S(z) + B(z)R(z) = A_m(z)A_0(z)$$

où A_m fixe la dynamique désirée et A_0 un polynôme observateur plus rapide.

Avantage par rapport au PID : le placement de pôles est exact, et l'action intégrale est intégrée structurellement dans S sans créer un triple pôle ($z=1$).

Premièrement, il est conseillé d'exécuter les cellules une par une, de regarder les résultats et de lire les questions sans chercher à y répondre immédiatement. Une fois le notebook pris en main, revenir sur les paramètres à modifier et regarder les résultats.

```
In [1]: from Models.BallBeam import ballbeam_config
from Models.BallBeam.StateSpace import LinearStateSpaceModel
from Models.BallBeam.TransferFunctions import TransferFunctionModel
from Models.BallBeam.NonlinearDynamics import NonlinearBallBeamModel

from Simulation.simulation import TFSimulator
from Simulation.simulation import HybridSim
from Simulation.simulation import NonLinearHybridSim

from Simulation.runners import *

from Metrics_Plotting.SimLog import SimLog
from Metrics_Plotting.Plotting import Plotting
from Metrics_Plotting.Metrics import Metrics

from Control.DiscretePID import DiscretePID
from Control.RSTController import RSTController
```

```

from Utils import computeRST
from Utils import utils

import numpy as np
import control as ct
import matplotlib.pyplot as plt

%matplotlib inline

```

Configuration de la simulation :

T est le temps total de simulation et dt est la période d'échantillonnage du correcteur.

```

In [2]: ballbeam_config.T=6
        ballbeam_config.dt=0.05

```

```

In [3]: X_0 = np.array([[0],[0]]) # État initial [positi
bb_TF = TransferFunctionModel(ballbeam_config) # Modèle en fonctions de trans
bb_sim_dis = TFSimulator(bb_TF.Tf_dis, X_0) # Simulateur discret (FT)
modele_NL = NonlinearBallBeamModel(ballbeam_config) # Modèle non linéaire
sim_NL = NonLinearHybridSim(modele_NL, ballbeam_config) # Simulateur non linéa
A = bb_TF.Tf_dis.den_list[0][0] # Dénominateur A(z) de la FT d
B = bb_TF.Tf_dis.num_list[0][0] # Numérateur B(z) de la FT dis

```

```

In [4]: ## Affichage des fonctions de transfert continue et discrète
print(bb_TF.Tf_cont)
print(bb_TF.Tf_dis)
print("Pôles discrets de la FT :", bb_TF.Tf_dis.poles())

```

```

<TransferFunction>: sys[0]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

```

```

0.21
----
s^2

```

```

<TransferFunction>: sys[0]$sampled
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']
dt = 0.05

```

```

0.0002625 z + 0.0002625
-----
z^2 - 2 z + 1

```

```

Pôles discrets de la FT : [1.+6.68896751e-09j 1.-6.68896751e-09j]

```

Questions :

- Quel est l'ordre de la fonction de transfert continue ? Et de la fonction discrète ?
- Les pôles discrets de la FT sont-ils à l'intérieur du cercle unité ? Qu'est-ce que cela signifie pour la stabilité en boucle ouverte ?

Modèle de référence \$A_m\$

La fonction de transfert désirée est un **second ordre continu** caractérisé par ω_0 (fréquence propre) et ζ (amortissement), discrétisé par correspondance de pôles $z_p = e^{s_p T_s}$.

Le gain statique $y(t \rightarrow \infty)$ de F_{des} doit être 1 ce qui garantit le suivi sans erreur de l'échelon.

```
In [5]: omega0 = 2.3      # fréquence propre désirée [rad/s]
        zeta  = 0.85     # coefficient d'amortissement

        omega_carr = omega0 * omega0
        Am_s = [(1/omega_carr), (2.0*zeta/omega0), 1.0]      # polynôme
        Am_d = np.poly(np.exp(np.roots(Am_s) * ballbeam_config.dt)) # pôles di

        Bm = sum(Am_d)                                       # numérateur
        Tf_des = ct.tf(Bm, Am_d, ballbeam_config.dt)         # FT de ré
```

```
In [6]: ## Affichage de la fonction de transfert désirée et de ses pôles
        print(Tf_des)
        print("Pôles continus s :", np.roots(Am_s))
        print("Pôles discrets z :", np.roots(Am_d))
        print("Rayon des pôles  :", np.abs(np.roots(Am_d)))
        print("Gain Statique      :", Tf_des.dcgain())
```

```
<TransferFunction>: sys[2]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']
dt = 0.05
```

```

          0.012
-----
      z^2 - 1.81 z + 0.8224
Pôles continus s : [-1.955+1.21160018j -1.955-1.21160018j]
Pôles discrets z : [0.90521201+0.05490493j 0.90521201-0.05490493j]
Rayon des pôles  : [0.90687559 0.90687559]
Gain Statique      : 1.0
```

La cellule suivante calcule la réponse indicielle de F_{des} à l'aide de la librairie `control`. Cette réponse sert de référence visuelle : la simulation en boucle fermée suit la même dynamique dominante (pôles de A_m), car le pôle de A_0 se simplifie avec le zéro correspondant dans le numérateur de H_{bf} . Un léger écart peut subsister en raison de la saturation de l'actionneur et du zéro de $B(z)$ absent dans F_{des} .

Questions :

- Le gain statique de F_{des} est-il bien égal à 1 ? Pourquoi est-ce indispensable pour la poursuite de consigne ? (utilisez le théorème de la valeur finale)
- Quelle est la valeur du rayon des pôles discrets ? Sont-ils stables ?
- Que se passe-t-il sur la rapidité de la réponse si l'on augmente ω_0 ?

```
In [7]: ## réponse indicielle de la fonction de transfert calculée par la librairie
t = np.arange(0, ballbeam_config.T, ballbeam_config.dt)
_, y_lib = ct.step_response(Tf_des, t)
```

```
In [8]: A0 = np.array([1, -0.5]) # pôle observateur à z=0.50 – doit être PLUS RAPIDE que L
```

Questions :

- Le pôle de A_0 ($z = 0.50$) est-il plus rapide ou plus lent que les pôles dominants de A_m ? Comment le vérifier ?
- Que se passerait-il si le pôle observateur était plus lent que les pôles dominants (rayon proche de 1) ?

Synthèse du correcteur RST

`Compute_Denominator_Matching_RST` résout l'équation diophantienne :

$$A(z)S(z) + B(z)R(z) = A_m(z) \cdot A_0(z)$$

Les polynômes dominants A_m et observateur A_0 sont passés séparément. Avec `Integrator=True`, le dénominateur effectif devient $A_{\text{eff}}(z) = A(z) \cdot (z-1)$. La fonction résout la diophantienne **directement** contre le polynôme caractéristique complet :

$$A_{\text{eff}}(z)\tilde{S}(z) + B(z)R(z) = A_m(z) \cdot A_0(z)$$

puis reconstruit $S(z) = (z-1)\tilde{S}(z)$ et calcule :

$$T(z) = t_0 \cdot A_0(z), \quad t_0 = \frac{A_m(1)}{B(1)}$$

Le gain t_0 garantit un gain statique unitaire. Le facteur A_0 dans T **annule le pôle observateur dans la fonction de transfert de référence** :

$$H_{\text{bf}} = \frac{B \cdot T}{A \cdot S + B \cdot R} = \frac{B \cdot t_0 \cdot A_0}{A_m \cdot A_0} = \frac{t_0 \cdot B}{A_m}$$

La réponse simulée suit donc T_{des} (second ordre seul), et non la boucle fermée d'ordre 3.

Rôle de `Integrator=True` : force $S(z) = (z-1)\tilde{S}(z)$ (regardez ce qu'il se passe avec la perturbation constante)

```
In [9]: S, R, T, H_cl = computeRST.Compute_Denominator_Matching_RST(Am_d, bb_TF.Tf_dis, True)
```

Direct Solve Used

--- RST synthesis complete ---

S: [0.68115888 -0.68115888]

R: [1214.63282869 -2231.57523706 1028.37032643]

T: [22.85583611 -11.42791806]

Exact closed-loop transfer function enforced by R,S,T:

Numerator coefficients: [0.00599966 0.00299983 -0.00299983]

Denominator coefficients: [1. -2.31042403 1.72763536 -0.41121167]

<TransferFunction>: sys[8]

Inputs (1): ['u[0]']

Outputs (1): ['y[0]']

dt = 0.05

0.006 z^2 + 0.003 z - 0.003

z^3 - 2.31 z^2 + 1.728 z - 0.4112

Closed-loop DC gain: 1.000000000000037

Closed-loop zeros (z): [-1. 0.5]

Closed-loop poles (z): [0.90521201+0.05490493j 0.90521201-0.05490493j 0.5 +0.j
]

Pole radii: [0.90687559 0.90687559 0.5]

Questions :

- Quel est l'ordre de H_{bf} ? Pourquoi est-il supérieur à celui de F_{des} ?
- Quel est l'effet de `Integrator=True` sur le polynôme $S(z)$? Quel facteur est ajouté ?
- Le gain statique de H_{bf} est-il bien 1 ? Tous les rayons des pôles sont-ils inférieurs à 1 ?
- Regardez les pôles et les zéros de H_{bf} , lesquels vont se simplifier, à quel polynôme correspondent-ils ?

Vérification de la synthèse

Après la synthèse, contrôler les trois points suivants :

1. **Gain statique de $H_{\text{bf}} = 1,0$** — confirme que la consigne est atteinte sans erreur statique de poursuite.
2. **Tous les rayons des pôles < 1** — confirme la stabilité de la boucle fermée.
3. **H_{bf} affiche un pôle à $z \approx 0.50$ (A0) et deux pôles dominants (Am)** — mais ce pôle A0 se simplifie avec le zéro correspondant dans le numérateur : la réponse simulée suit le 2nd ordre de F_{des} .

Marges de stabilité

La boucle ouverte est $L(z) = B(z) \backslash R(z) \backslash \backslash \backslash \bigl[A(z) \backslash S(z) \bigr]$.

```
In [10]: L_rst = bb_TF.Tf_dis * ct.tf(np.squeeze(R.num[0][0]), np.squeeze(S.num[0][0]), ball
      metriques = Metrics()
      metriques.Stability(L_rst)
```

Gain margin: -23.0 dB (×0.07)

Phase margin: 79.0°

Questions :

- La marge de gain est-elle supérieure à 6 dB ? La marge de phase est-elle supérieure à 45° ?
- Comparer ces marges avec celles obtenues avec le correcteur PID dans le notebook précédent. Lequel est plus robuste ?

```
In [11]: freq_max= np.pi/ballbeam_config.dt ## calcul de la fréquence maximale
      ct.bode_plot(L_rst, omega_limits=[0.1,freq_max-10]) ## -10 ajouté pour des raisons
```

Out[11]: <control.ctrlplot.ControlPlot at 0x1a36b737380>

Notes sur la marge de gain et la fréquence maximale:

Comme montré sur le diagramme de Bode, la fonction de transfert contient trois pôles $z=1$, c'est un triple intégrateur (si `Integrator=True`, sinon c'est un double).

Le diagramme de Bode commence donc à $(-270^\circ$ ou $-180^\circ)$, et la marge de gain est "inversée". Cependant elle peut être interprétée comme si elle était positive (dans cette situation).

La fréquence maximale peut être expliquée par les caractéristiques d'un système échantillonné. La relation $z = e^{j\omega dt}$ impose que $-\pi < \omega dt < \pi$ pour qu'à une fréquence d'échantillonnage corresponde une valeur $z \in \mathbb{C}$. Au dessus de $\omega_{\max} = \frac{\pi}{dt}$, plusieurs ω correspondent au même z et les calculs n'ont plus de sens.*

Vous pouvez tester ça en ajoutant 1000 à `freq_max`

Simulation

On crée une instance du **contrôleur** RST initialisé avec les **polynômes** R, S, T et on utilise le simulateur discret. Le premier True/False détermine la présence d'une **perturbation d'entrée en échelon** ($D = 2$, constante à partir de $t = 3$ s) injectée à l'entrée de la plante — la plante reçoit $u + d$ tandis que la commande journalisée est la sortie brute du correcteur. Le deuxième active (si True) la saturation de l'actionneur.

```
In [12]: rst_controller = RSTController(R, S, T)
      logger_rst = SimLog()

      reference = 1.0
```

```
y_0 = 0.0
```

```
logger_rst = run_discrete_control(bb_sim_dis, rst_controller, ballbeam_config, refe
```

Métriques de performance

Dépassement, temps de montée (10 %→90 %) et temps d'établissement (bande ±10 %) calculés sur la phase de suivi de consigne (avant perturbation).

```
In [13]: metriques.response_data(logger_rst, reference)
```

```
Dépassement :          1.1 %
Temps de montée :      0.750 s (10 %→90 %)
Temps d'établissement : 1.050 s (bande ±10 %)
```

Questions :

- Comparer le dépassement et le temps de montée obtenus ici avec ceux du correcteur PID. Lequel est plus rapide ? Lequel dépasse davantage ?
- Le dépassement observé est-il cohérent avec le coefficient d'amortissement $\zeta = 0.70$ choisi pour A_m ? (utiliser l'équation de)

Rejet de perturbation d'entrée en échelon

La fonction de transfert **perturbation d'entrée** → **position** est :

$$F_p(z) = \frac{B(z)S(z)}{A(z)S(z) + B(z)R(z)} = \frac{B(z)S(z)}{A_m(z)A_0(z)}$$

La perturbation est un **échelon constant** $D = 2$ [rad] injecté à l'entrée de la plante à partir de $t = 3$ s (erreur de modèle, force extérieure constante). C'est le type de perturbation le plus discriminant pour comparer RST et PID.

Erreur statique (théorème de la valeur finale, $z = 1$, $A(1) \neq 0$) :

$$F_p(1) = \frac{B(1)S(1)}{A(1)S(1) + B(1)R(1)} = \frac{S(1)}{R(1)}$$

```
In [14]: A_arr = np.squeeze(A)
B_arr = np.squeeze(B)
R_arr = np.squeeze(R.num[0][0])
S_arr = np.squeeze(S.num[0][0])

# Perturbation d'entrée → position : Fp_in = B·S / (A·S + B·R)
num = np.polymul(B_arr, S_arr)
den = np.trim_zeros(np.round(np.polyadd(np.polymul(A_arr, S_arr), np.polymul(B_arr,
Gd = ct.tf(num, den, ballbeam_config.dt)
print(Gd)
print(f"Gain statique Fp_in(z=1) : {Gd.dcgain():.6f} (0 = rejet exact)")

t = np.asarray(t).flatten()
```

```
_, d_resp = ct.step_response(Gd, t)
d_resp = np.squeeze(d_resp)
```

```
<TransferFunction>: sys[13]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']
dt = 0.05
```

```
0.0001788 z^2 + 6.05e-16 z - 0.0001788
```

```
-----
z^3 - 2.31 z^2 + 1.728 z - 0.4112
```

```
Gain statique Fp_in(z=1) : 0.000000 (0 = rejet exact)
```

Affichage des résultats

Trois graphiques sont tracés :

1. **Position réelle $y_{\text{plant}}(t)$** — après $t = 3$ s la perturbation d'entrée $D = 2$ agit en permanence.
2. **Commande $u(t)$** — signal envoyé à l'actionneur (sans d).
3. **Réponse indicielle de $D \cdot F_p^{in}(z)$** — montre la déviation prédite, mais n'est pas liée à la réponse simulée

```
In [ ]: _, y_lib_Hcl = ct.step_response(H_cl, t)  # réponse de la boucle fermée réelle (av

D = 2.0 # magnitude de la perturbation d'entrée

fig, ax = plt.subplots(3, 1, figsize=(10, 8), sharex=True)

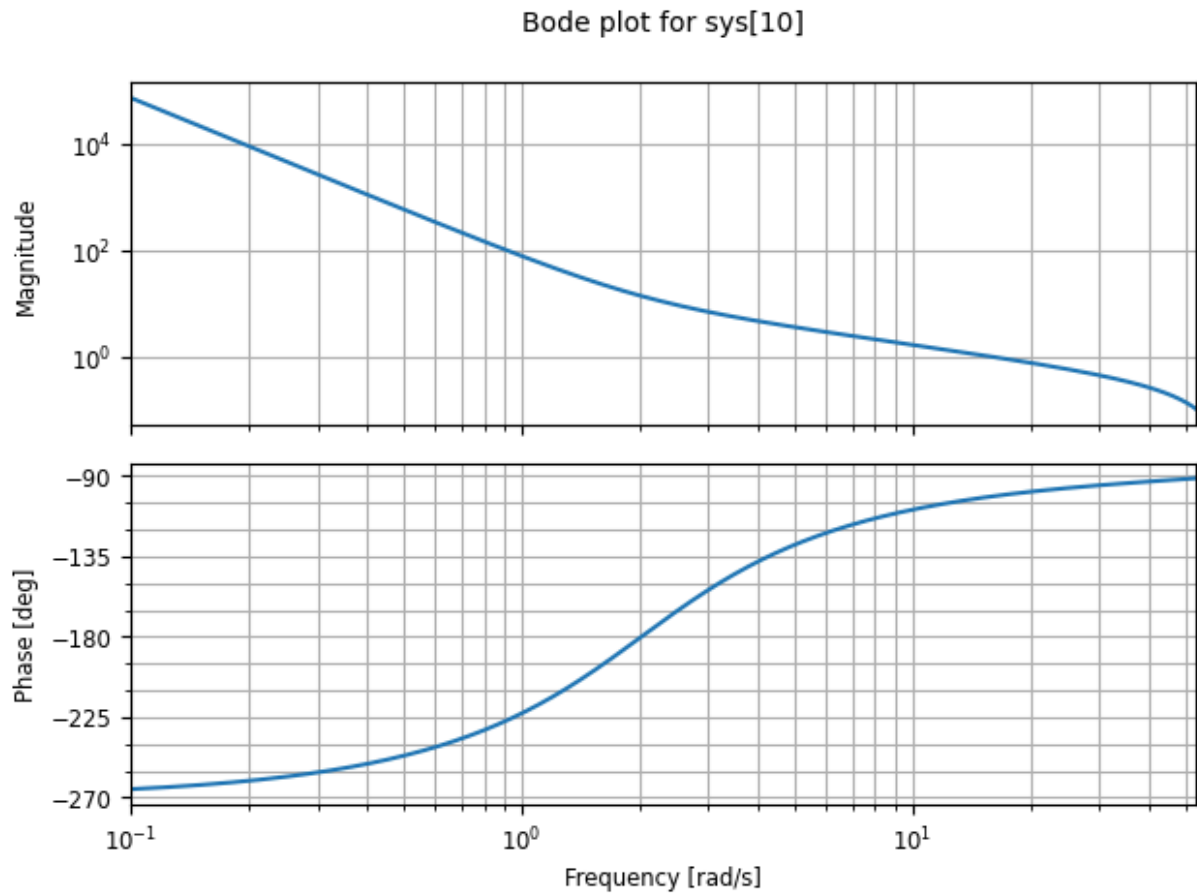
ax[0].step(logger_rst.t_hist, logger_rst.y_hist, label='y(t) - position')
ax[0].step(t, np.squeeze(y_lib), linestyle=':', color='gray', label='Tf_d
ax[0].axhline(y=reference, color='green', linestyle='--', linewidth=1, label=f'Cons
ax[0].axvline(x=3, color='orange', linestyle='--', linewidth=1, label="Perturbation
ax[0].set_title("Position y(t) - RST")
ax[0].set_ylabel("Position [m]")
ax[0].legend()
ax[0].grid()

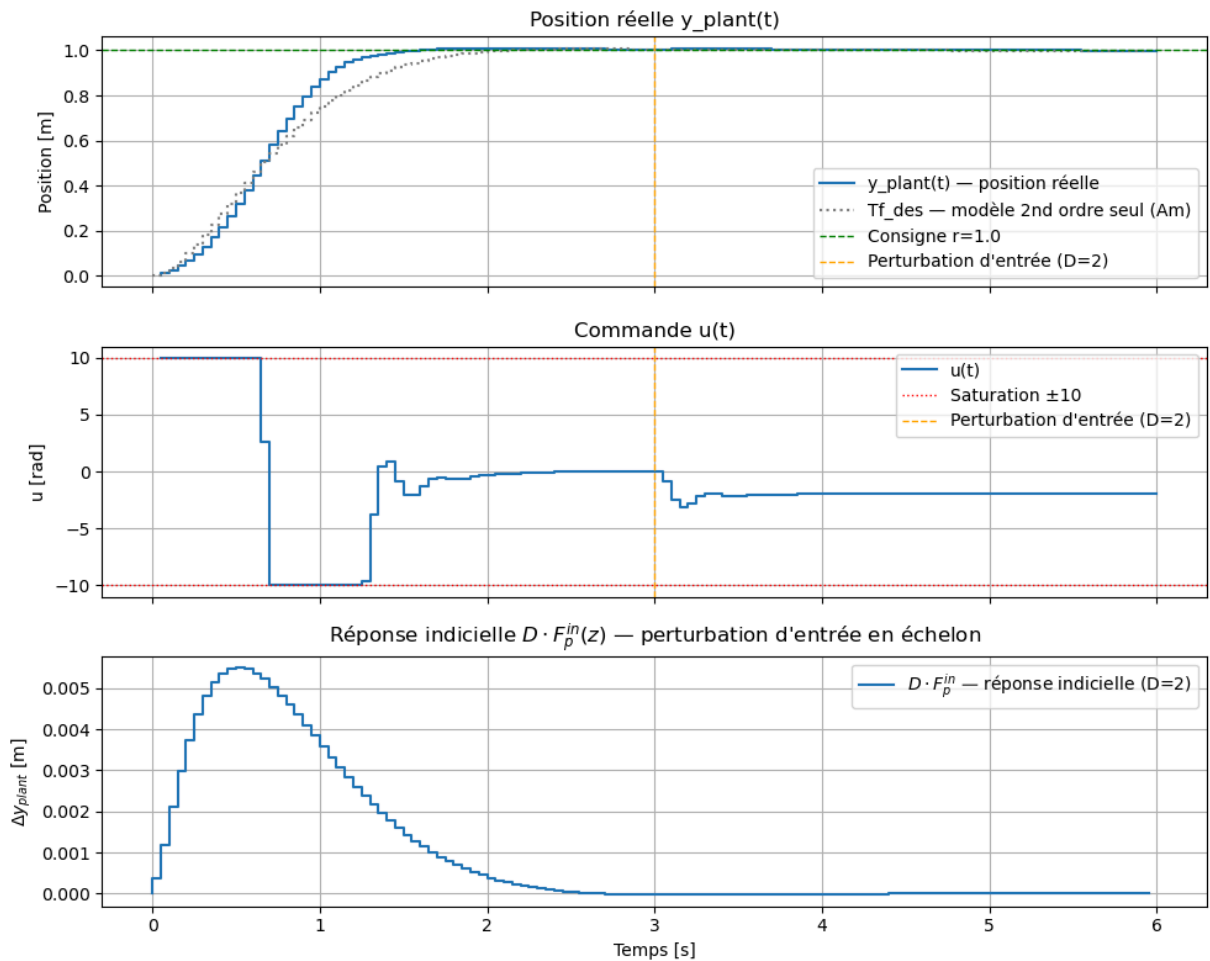
ax[1].step(logger_rst.t_hist, logger_rst.u_hist, label='u(t)')
ax[1].axhline(y= 10, color='red', linestyle=':', linewidth=1, label='Saturation ±10
ax[1].axhline(y=-10, color='red', linestyle=':', linewidth=1)
ax[1].axvline(x=3, color='orange', linestyle='--', linewidth=1, label="Perturbation
ax[1].set_title("Commande u(t)")
ax[1].set_ylabel("u [rad]")
ax[1].legend()
ax[1].grid()

ax[2].step(t, D * d_resp, label="$D \cdot F_p^{in}$ - réponse indicielle (D=2)")
ax[2].set_title("Réponse indicielle $D \cdot F_p^{in}(z)$ - perturbation d'entrée
ax[2].set_ylabel("$\Delta y_{\text{plant}}$ [m]")
ax[2].set_xlabel("Temps [s]")
ax[2].legend()
ax[2].grid()
```



```
plt.tight_layout()  
## Décommenter pour enregistrer la figure  
#plt.savefig("Commande_RST_figure1.jpg")  
plt.show()
```





Questions :

- La position réelle simulée suit-elle bien T_{des} (courbe grise pointillée) avant $t = 3$ s ?
- Après la perturbation d'entrée $D = 2$ à $t = 3$ s, la position revient-elle **exactement** à la consigne r ? Pourquoi ?
- Sur la réponse indicielle de $D \cdot F_p^{\text{in}}$, la courbe tend-elle bien vers 0 ?
- La commande $u(t)$ sature-t-elle après la perturbation ? Quelle est la valeur de u_{ss} à l'état stationnaire ?

Comparaison RST vs PID — rejet de perturbation d'entrée en échelon (bande passante identique)

Questions :

- L'erreur statique du RST est-elle bien nulle après la perturbation ? Celle du PID correspond-elle à D/K_p ?
- Quelle propriété de $S(z)$ garantit le rejet exact ($S(1) = 0$) ? Comment cela apparaît-il dans l'équation diophantienne ?
- Que montre le gain statique $F_p^{\text{in}}(1) = S(1)/R(1)$ calculé ci-dessous pour RST et PID ?

```
In [16]: from Control.DiscretePID import DiscretePID

# Recréer Le PID (mêmes gains que dans Le notebook PID) pour La comparaison
pid_cmp = DiscretePID(43, 0, 21, ballbeam_config.dt, "filtered")
R_pid_arr = np.squeeze(pid_cmp.R.num[0][0])
S_pid_arr = np.squeeze(pid_cmp.S.num[0][0])

# Fp_in PID = B·S_pid / (A·S_pid + B·R_pid)
num_pid = np.polymul(B_arr, S_pid_arr)
den_pid = np.polyadd(np.polymul(A_arr, S_pid_arr), np.polymul(B_arr, R_pid_arr))
Fp_pid = ct.tf(num_pid, den_pid, ballbeam_config.dt)

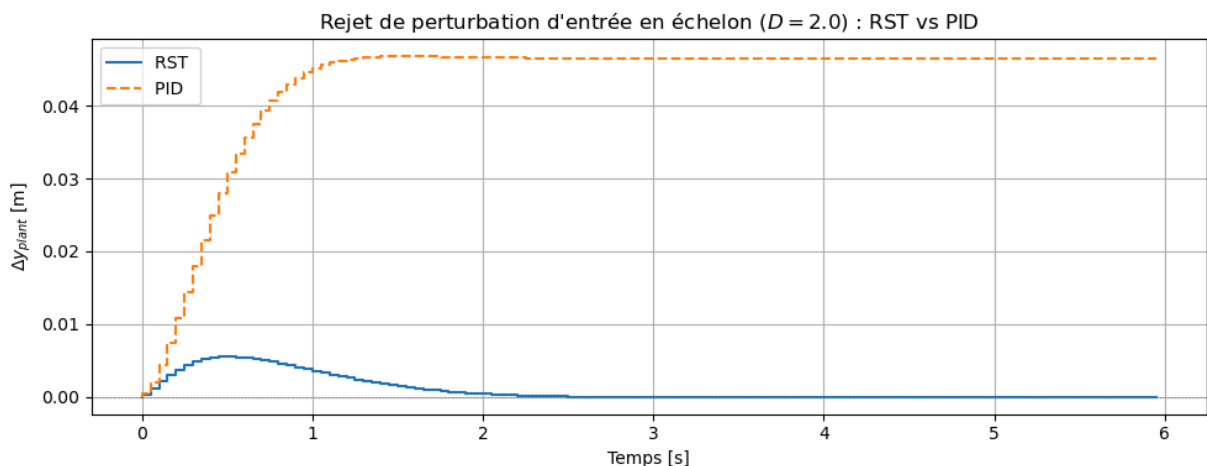
print(f"Gain statique RST  Fp_in(1) = {Gd.dcgain():.6f} ")
print(f"Gain statique PID  Fp_in(1) = {Fp_pid.dcgain():.6f} ")

_, d_resp_pid = ct.step_response(Fp_pid, t)
d_resp_pid = np.squeeze(d_resp_pid)

fig, ax = plt.subplots(figsize=(10, 4))
ax.step(t, D * d_resp, label=f'RST ')
ax.step(t, D * d_resp_pid, label=f'PID ', linestyle='--')
ax.axhline(0, color='gray', linewidth=0.8, linestyle=':', alpha=0)
ax.set_title(f"Rejet de perturbation d'entrée en échelon ($D = {D}$) : RST vs PID")
ax.set_ylabel("$\\Delta y_{\\text{plant}}$ [m]")
ax.set_xlabel("Temps [s]")
ax.legend()
ax.grid()
plt.tight_layout()
plt.show()
```

Gain statique RST Fp_in(1) = 0.000000

Gain statique PID Fp_in(1) = 0.023256



Simulation non linéaire

Même configuration que la simulation linéaire, mais le procédé est remplacé par la **dynamique non linéaire** de la balle sur poutre, intégrée par RK4 à $\Delta t_{\text{plant}} = 1 \times 10^{-3}$ s.

Différences attendues par rapport à la simulation linéaire :

- La saturation à ± 10 peut retarder la compensation de la perturbation d'entrée.
- Les non-linéarités de $\sin\alpha$ peuvent modifier légèrement la dynamique de rejet.

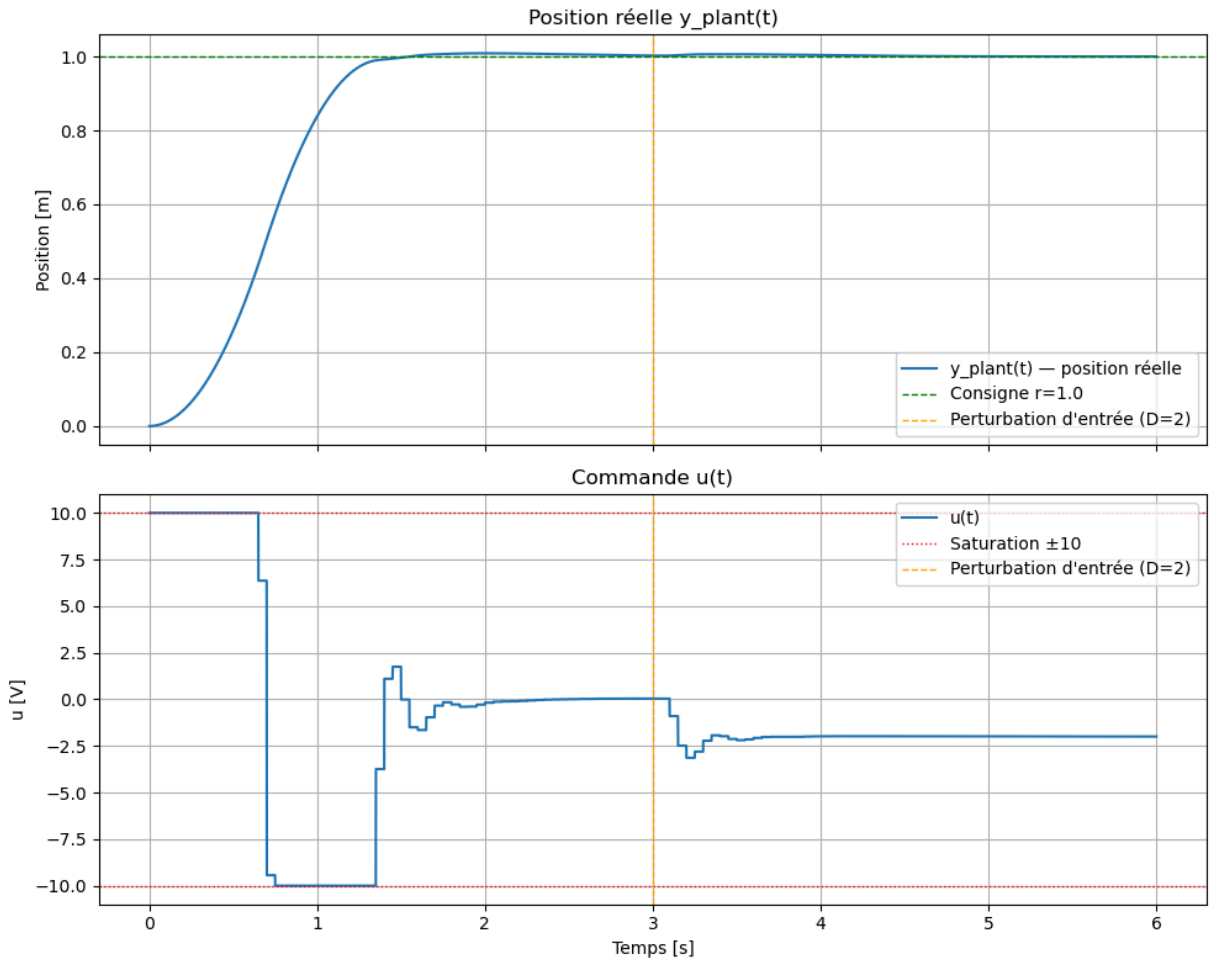
```
In [17]: logger_NL = SimLog()
logger_NL = run_continuous_control_loop(sim_NL, rst_controller, reference, X_0, log
```

```
In [ ]: fig, ax = plt.subplots(2, 1, figsize=(10, 8), sharex=True)

ax[0].plot(logger_NL.t_hist, logger_NL.y_hist, label='y(t)')
ax[0].axhline(y=reference, color='green', linestyle='--', linewidth=1, label=f'Cons
ax[0].axvline(x=3, color='orange', linestyle='--', linewidth=1, label="Perturbation
ax[0].set_title("Position y(t)")
ax[0].set_ylabel("Position [m]")
ax[0].legend()
ax[0].grid()

ax[1].plot(logger_NL.t_hist, logger_NL.u_hist, label='u(t)')
ax[1].axhline(y= 10, color='red', linestyle=':', linewidth=1, label='Saturation ±10
ax[1].axhline(y=-10, color='red', linestyle=':', linewidth=1)
ax[1].axvline(x=3, color='orange', linestyle='--', linewidth=1, label="Perturbation
ax[1].set_title("Commande u(t)")
ax[1].set_ylabel("u [rad]")
ax[1].set_xlabel("Temps [s]")
ax[1].legend()
ax[1].grid()

plt.tight_layout()
## Décommenter pour enregistrer la figure
#plt.savefig("Commande_RST_figure2.jpg")
plt.show()
```



```
In [19]: metriques.response_data(logger_NL,reference)
```

Dépassement : 0.9 %
 Temps de montée : 0.778 s (10 %→90 %)
 Temps d'établissement : 1.088 s (bande ±10 %)

Questions :

- Le rejet de la perturbation d'entrée est-il différent entre la simulation linéaire et non linéaire pour le RST ?
- La commande $u(t)$ diffère-t-elle entre les deux simulations après la perturbation ? Quelle est la valeur de u_{ss} ?
- L'erreur statique du RST est-elle encore nulle dans la simulation non linéaire ? Pourquoi le résultat devrait-il être le même ?

Tests de paramètres

Relancez les cellules de configuration et de synthèse après avoir modifié un paramètre. Observez l'effet sur les marges de stabilité, les métriques et les courbes de simulation.

1. Fréquence propre désirée ω_0

ω_0 [rad/s]	Rapidité attendue	Dépassement attendu
2	Lente	Faible
4 (défaut)	Nominale — alignée sur le PID	Nominal
6	Rapide	Élevé

Modifier `omega0` dans la cellule du modèle de référence. Que devient la marge de gain quand ω_0 augmente ?

2. Coefficient d'amortissement ζ

ζ	Comportement
0.50	Fortement sous-amorti (oscillations)
0.70	Légèrement sous-amorti
0.85 (défaut)	Proche critique — aligné sur le PID
1.00	Critique (pas de dépassement)

Modifier `zeta`. Observer le dépassement dans la simulation linéaire et comparer avec la valeur théorique $D\% \approx e^{-\pi\zeta/\sqrt{1-\zeta^2}} \times 100\%$.

3. Pôle observateur A_0

Pôle A_0 (z)	Rapidité de l'observateur
0.95	Trop lent (proche des pôles dominants)
0.50 (défaut)	Rapide — bien séparé des pôles dominants
0.20	Très rapide (peut amplifier le bruit de mesure)

Modifier `A0 = np.array([1, -z_obs])`. Vérifier que tous les pôles de H_{bf} restent bien à l'intérieur du cercle unité.

4. Action intégrale (Integrator)

Integrator	Effet
True (défaut)	$S(z)$ contient $(z-1)$
False	Pas d'intégrateur

Passer `Integrator=False` dans l'appel à `Compute_Denominator_Matching_RST`. Observer la courbe de rejet de perturbation $F_p(z)$: le gain statique de F_p est-il encore 0 ?

5. Consigne `reference`

<code>reference</code> [m]	Observation
0.5	Faible déplacement, pas de saturation
1.0 (défaut)	Nominal
2.0	Saturation prolongée, non-linéarités plus visibles

Modifier `reference` dans la cellule de simulation. Comparer les simulations linéaire et non linéaire pour une grande consigne.